MOVIE RECOMMENDATION SYSTEM

Machine Learning • Recommendation Systems • Content-Based Filtering

Student: Kavyansh Bhardwaj
Institution: SIMS Delhi
Course / Drive: Artificial Intelligence and Internship Drive
Internship Organization: Naviotech
Submission Deadline: 30 October 2026

Academic project report
Prepared as a working educational prototype. The source code and supplied starter data should be tested and extended before final assessment.

1. Abstract

This project demonstrates a content-based movie recommendation system. It combines movie genres and plot summaries, converts the text into TF-IDF vectors, and ranks similar titles using cosine similarity. A Streamlit interface allows a user to select a title and request recommendations. The supplied catalogue contains 30 curated demonstration records; it should be expanded before drawing conclusions about recommendation quality at scale.

2. Problem statement

A large catalogue can make it difficult for viewers to discover films they may enjoy. This project ranks titles that share descriptive content with a selected movie. It is a content-based prototype: it does not learn from an individual's viewing history, ratings or behaviour.

3. Objectives

• Prepare a structured movie catalogue containing title, genres and overview.

• Combine descriptive fields into text features.

• Transform text into TF-IDF vectors.

• Calculate cosine similarity between movie vectors.

• Return and explain the top-N most similar titles through a web interface.

4. System architecture and workflow

Movie catalogue → combine genres and overview → TF-IDF vectorizer → cosine similarity matrix → exclude selected title → rank by similarity → display top-N recommendations.

5. Technology stack

Technology	Role
Python	Core programming language
Pandas	Load and manage movie metadata
Scikit-learn	TF-IDF feature extraction and cosine similarity
TF-IDF	Represent genres and plot descriptions as weighted vectors
Cosine similarity	Rank titles by vector similarity
Streamlit	Interactive browser-based interface

6. Implementation guide

1. Install Python 3.10 or newer.

2. Open the project folder in VS Code or a terminal.

3. Create and activate a virtual environment: `python -m venv venv` then `venv\Scripts\activate` on Windows.

4. Install dependencies: `pip install -r requirements.txt`.

5. Run the app: `streamlit run app.py`.

6. Select a movie and choose the number of recommendations.

7. Expand the 30-title demonstration catalogue with a properly attributed dataset such as MovieLens.

7. Advantages

• Simple and interpretable recommendation method.

• Does not require user profiles or historical ratings.

• Can recommend less-popular catalogue titles if metadata is descriptive.

• Easy to extend with posters, filters and a larger catalogue.

8. Disadvantages and limitations

• Recommendations depend on the quality and coverage of movie metadata.

• No user history or rating signals are used.

• Similar descriptions can produce repetitive recommendations.

• Similarity scores are not probabilities, quality ratings or guarantees of user preference.

9. Practical applications

• Movie discovery on streaming or entertainment catalogues.

• Personalized browsing prototypes when user preferences are later added.

• Educational demonstration of information retrieval and recommendation algorithms.

10. Future scope

• Expand the catalogue and add poster and release metadata with proper licensing.

• Add collaborative filtering using user-item ratings.

• Build a hybrid recommender combining content and rating signals.

• Add user feedback, genre filters and diversity-aware ranking.

11. Conclusion

The project demonstrates a complete content-based recommendation pipeline, from movie metadata and TF-IDF vectors to cosine-similarity ranking and a browser-based interface. The 30-title catalogue is a teaching scaffold rather than a broad commercial catalogue. A stronger submission should expand the data, evaluate recommendation relevance and explain the limits of content-only ranking.

12. References

• GroupLens Research. MovieLens datasets: https://grouplens.org/datasets/movielens/.

• Scikit-learn documentation: https://scikit-learn.org/.

• Streamlit documentation: https://docs.streamlit.io/.

Academic integrity and result reporting: Run the application yourself, add screenshots from your own run, record actual measurements, and acknowledge any external datasets or code used. Do not invent accuracy, user-study results, or deployment claims.